Estruturas de Dados com C++

Jonathan Tenório de Lima

25 de março de 2026

Sumário

1 Ordenação **2**
1.1 Insertion sort 2
1.2 Bubble sort 3
1.3 Selection sort 5

1 Ordenação

1.1 Insertion sort

Esse algoritmo de ordenação funciona semelhante a organizar as cartas do baralho na mão, durante uma partida. Inicialmente, as cartas podem estar desordenadas, então, você seleciona a segunda e compara com aquelas à esquerda, movendo a carta até a posição em esta seja a menor da sequência. A cada intervalo de interação, a carta de comparação pode ser chamada de ***carta*** e o vetor contendo as cartas será denominado $\boldsymbol{v}$. A figura 1 exemplifica o funcionamento do método de ordenação. A cada tempo, a *carta* selecionada é deslocada até a posição correspondente ordenada, o que "empurra"as outras cartas para uma posição mais adiante.

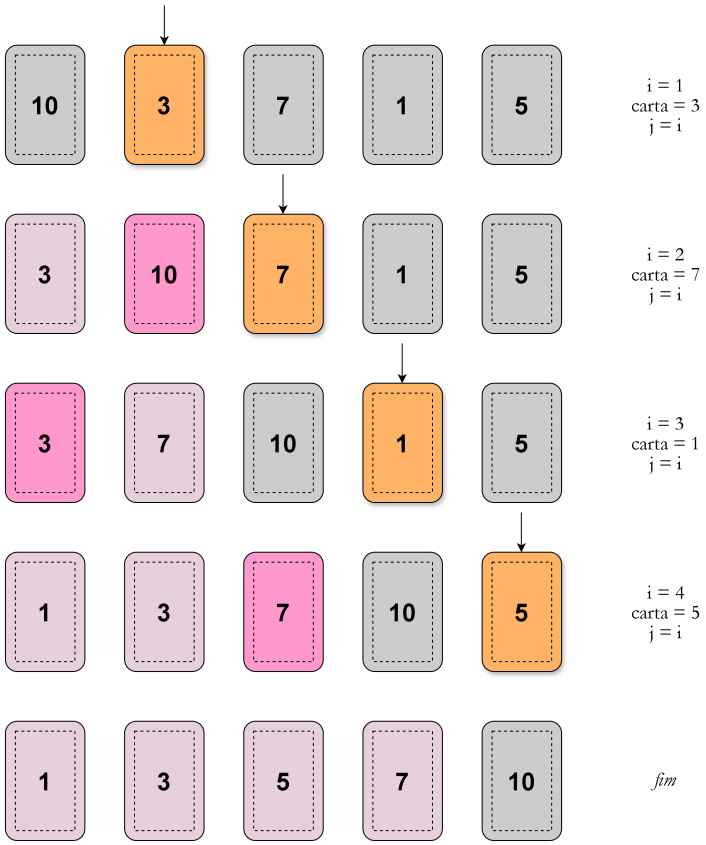

Figura 1: Exemplo do funcionamento do Insertion Sort

Assumindo que o vetor v possui n posições [1, 2, 3, ..., n], o pseudocódigo deste algoritmo pode ser dado como a seguir:

Algorithm 1 Insertion Sort

```
function INSERTIONSORT(v)
    n ← tamanho(v)
    for i ← 2 to n do                    ▷ A ordenação começa na posição 2 do vetor
        carta ← v[i]                     ▷ A "carta"será sempre a posição atual do vetor
        j ← i                            ▷ O índice j é utilizado para comparar
        while j > 0 and v[j − 1] > carta do
            v[j] ← v[j − 1]
            j ← j − 1
        end while
        v[j] ← carta                     ▷ Se j mudou, a "carta"vai para a esquerda
    end for
end function
```

Também é possível escrever o código correspondente em C++ como a seguir:

```cpp

void insertionSort( std::vector<int>& v ) {
    for ( int i = 1; i < n; ++i ){
        auto carta = v[i];
        auto j = i;
        while ( j > 0 and v[j-1] > carta ) {
            v[j] = v[j-1];
            j = j - 1;
        }
        v[j] = carta;
    }
}
```

Observações importantes:

- Os vetores em C++ iniciam na posição 0 (e não 1, como admitido no pseudocódigo);
- A função *insertionSort(v)* não retorna nada, apenas modifica o estado do vetor usado como argumento, por isso, é do tipo *void*;
- O índice j deve ser definido dentro do *for* e fora do *while*, pois, caso ocorra mudança no valor, está será a nova posição da *carta*.

1.2 Bubble sort

O algoritmo de ordenação *bubble sort* segue um padrão de comparações dois a dois entre os elementos do vetor, sempre orientando para que os maiores elementos sejam "*empurrados*"para posições posteriores do vetor. Alguns comentam que esse algoritmo tenta copiar o movimento de bolhas (*bubbles*) em uma garrafa com líquido, onde o empuxo conduz as bolhas para posições mais elevadas da garrafa, sempre no sentido da interface líquido-ar.

Este algoritmo é reconhecido pela simplicidade e também por ser intuitivo (talvez o termo mais correto seja *naive*). Geralmente é apresentado em quatro versões, com refinamentos sucessivos, como apresentado pelo prof. Mário Felice (link). O pseudocódigo a seguir é baseado na quarta versão do algoritmo.

Algorithm 2 Bubble Sort

```
function BUBBLESORT(v)
    rlim ← v.size()                          ▷ Define o limite a direita (right limite, rlim
    for i ← 1 to n do
        lsp ← 1                              ▷ Define a última posição de troca, lsp
        for i ← 2 to rlim do
            if v[i − 1] > v[i] then
                aux ← v[i − 1]
                v[i − 1] ← v[i]
                v[i] ← aux
                lsp ← i                      ▷ lsp recebe a última posição em que ocorreu troca
            end if
        end for
        rlim ← lsp                           ▷ Define o novo limite à direita como a última posição de troca
    end for
end function
```

Observações úteis:

- A variável *rlim* significa *right limite* ou limite direito e indica a posição máxima do vetor para comparação a cada *loop*. Isto ocorre, porque a cada iteração, as posições $[x_j, x_{j+1}, ..., n]$ já estarão ordenadas.
- O limite direito (*right limit*), *rlim* é definido como a última posição do vetor que foi ordenada na iteração anterior. Note que inicialmente $rlim = n$ e vai diminuindo a cada iteração.
- A última posição em que ocorreu troca de posição (*last switch position*), *lsp*, é definida como a posição i obtida ao final de cada condicional.
- O *lsp* inicia o *loop* mais externo com valor "1", pois pressupõe-se que ocorrerá troca já na primeira posição do vetor. Entretanto, esta é apenas uma convenção.
- O *loop* interno começa sempre na segunda posição do vetor, pois o elemento da posição i será comparado com o elemento da posição *i-1*.

```cpp

void bubbleSort( std::vector<int>& v ) {

    int rlim = v.size();  // limite direito

    for ( std::size_t j = 0; j < v.size(); ++j ) {
            std::size_t lsp = 0;
            // lsp: last swap position
            for ( auto i = 1; i < rlim; ++i ) {
                if ( v[i-1] > v[i] ) {
                    auto aux = v[i-1];
                    v[i-1] = v[i];
                    v[i] = aux;
                    lsp = i;
                }
```

```
            }
            rlim = lsp;
        }
```

1.3 Selection sort

Algorithm 3 Selection Sort

procedure SELECTIONSORT(v)
  $n \leftarrow v.size()$
  for $i \leftarrow 1$ to $n - 1$ **do**
    $idm \leftarrow i$ ▷ Atribui o índice mínimo à posição i
    for $j \leftarrow i + 1$ to n **do** ▷ As posições anteriores já estão ordenadas
      if $v[j] < v[idm]$ **then**
        $idm \leftarrow j$ ▷ Localiza o índice do menor elemento a cada loop
      end if
    end for
    $aux \leftarrow v[i]$
    $v[i] \leftarrow v[idm]$
    $v[idm] \leftarrow aux$
  end for
end procedure

```

void selectionSort( std::vector<int>& v ) {

    for (std::size_t i = 0; i < v.size() - 1; ++i){
            int idx_min = i;
            for (std::size_t j = i + 1; j < v.size(); ++j){

                /*
                    Nesta parte do código é realizada uma comparação entre o elemento atual 'j'
                    e o elemento correspondente ao índice mínimo 'idx_min'. Caso o elemento da
                    posição 'j' seja menor, então, o idx_min recebe a posição 'j'. Este teste
                    é conduzido desde 'i+1' (equivalente a 1, quando i = 0) até o último elemento
                    do vetor 'v'.
                */
                if (v[j] < v[idx_min]) {   // O sinal de menor ordena em ordem crescente
                    idx_min = j;
                }
            }
            /*
                Depois que o novo índice do elemento mínimo é definido 'idx_min', a posição 'i'
```

```
                atual recebe o valor mínimo e já está ordenada. As próximas interações começam
                a partir da posição seguinte.
            */
            auto aux = v[i];
            v[i] = v[idx_min];
            v[idx_min] = aux;
        }
```